

Does a benchmark score measure the model, or the scaffold around it?

Santi Bent

Benchmark scores get reported as properties of models. A paper says a model scores 0.83 on some benchmark, and readers take that as a fact inherent in the model. But every score is produced by a model and a scaffold together, meaning the prompting, the number of calls, and the way the answer is assembled. The scaffold is rarely held constant across papers and is often not described at all.

If the scaffold itself moves the number, then comparisons across papers are not comparing what they claim to compare. That matters more than usual for dual-use biology benchmarks, because those numbers feed capability assessments and model release decisions.

I ran a small study to see how much the scaffold moves the number. Same model (Haiku 4.5), same questions, 2 different scaffolds, each run twice.

What I did

The benchmark I used was LAB-Bench ProtocolQA, 108 multiple-choice questions about laboratory protocols. The option count is not fixed. Each item is built from the correct answer, a variable number of distractors, and an explicit "insufficient information to answer the question" choice, giving 5 options for 77 items, 6 for 21, 7 for 9, and 8 for 1. Chance performance is therefore item-dependent and averages about 0.188 rather than a flat 0.20. I used the existing implementation in `inspect_evals` without modifying the dataset or the scorer. The model is `claude-haiku-4-5`.

The first scaffold is a plain single call. The model sees the question and picks an answer. This is what most reported benchmark numbers come from.

The second scaffold breaks each question into up to 3 sub-questions, answers each one separately, then reads those answers and picks a final option. The important detail is that each sub-question gets its own fresh context. The call for sub-question two contains only sub-question two. It has no access to sub-question one or its answer, not because anything was cleared, but because it was never given. The only thing that crosses between stages is the text of the sub-answers, pasted into the final prompt as data rather than carried forward as conversation.

Design details that constrain interpretation

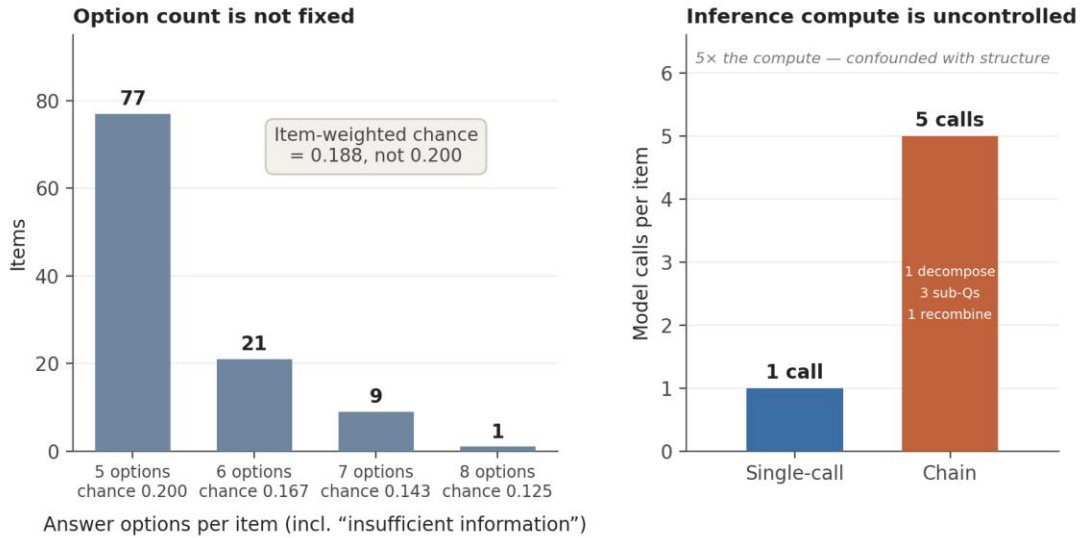


Figure 1. Study design. Left: distribution of answer-option counts across the 108 items and the resulting per-item chance rate. Right: model calls per item under each scaffold.

I ran each scaffold twice with independent sampling at temperature = 1.0.

Two things vary from run to run, not one. `inspect_evals`'s `retrieve_hf_dataset` defaults to `shuffle_choices=True` with no seed, and I did not override that default, so each of the four runs drew a fresh random permutation of every item's answer options (Inspect remaps the target letter, so scoring is unaffected). The within-arm noise floor therefore includes both temperature-1 sampling and option-order re-randomization, and the two arms saw a given item's options in different orders. This does not bias either arm, both draw permutations the same way, but it means the between-arm disagreement cannot be attributed to scaffold structure alone, and the noise floor is wider than a sampling-only account would predict.

The second run of each arm is the part that makes the rest interpretable. Two runs of the same scaffold will not agree perfectly, because on many of these items the model is only weakly confident, so either temperature-1 sampling or the independent per-run reshuffling of the answer options can change which one it picks. If I only compared one run of each scaffold, I would have no way to tell whether any disagreement I found was about the scaffold or was just the noise any two runs would show. So I measured how much each scaffold disagrees with itself before asking how much the two scaffolds disagree with each other.

I also scored abstentions separately from wrong answers throughout. A model declining to commit is not the same event as a model answering incorrectly, and folding them together destroys information. That decision turned out to matter.

Worth being precise about what an abstention is here, because it constrains what the result means. ProtocolQA includes "insufficient information to answer the question" as an answer choice on every

item, and the scorer treats selecting it as a non-answer rather than a wrong answer. My instrumentation counts a sample as an abstention if the model selected that choice or if the completion had no parseable answer line at all. The second case never occurred. Across all 432 samples in the four runs, the parse-failure path fired zero times, so every abstention reported below is the model deliberately picking an option the benchmark handed it. This is in-band abstention on a forced-choice item, not a model refusing a request.

Total cost for all four runs was: \$4.24.

Results

Accuracy did not move: The single-call scaffold scored 0.528 and 0.556 across its two runs. The chain scaffold scored 0.491 and 0.528. All four numbers sit inside a range that four runs at this sample size cannot separate. There is no accuracy effect here. These figures are computed over all 108 items, with abstentions counted as wrong, which is Inspect's default and the number a paper would report.

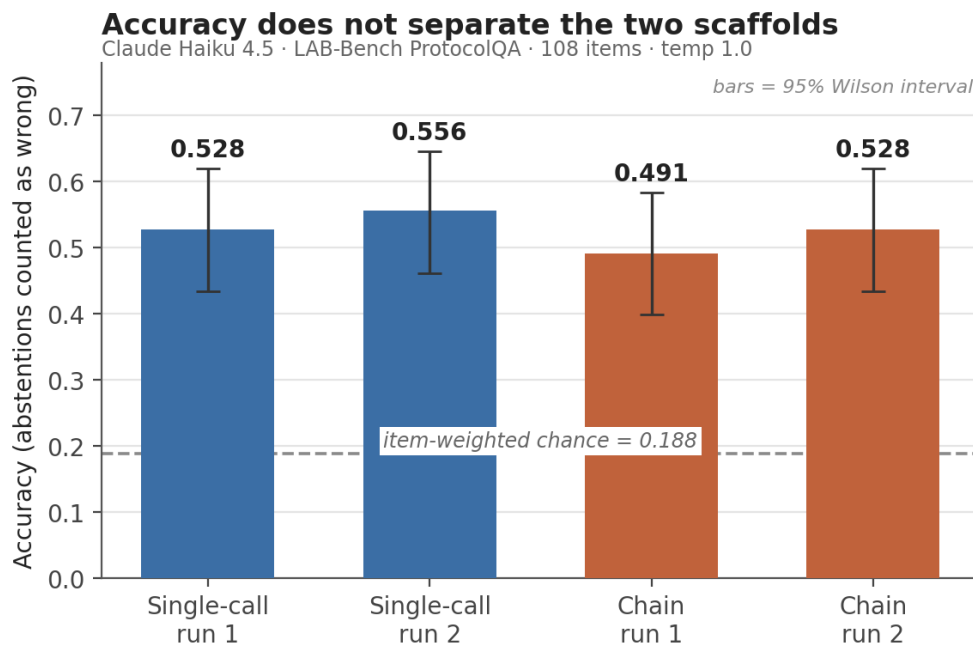


Figure 2. Accuracy by scaffold and run, with 95% normal-approximation intervals (Inspect's per-run standard error $\times 1.96$) and the item-weighted chance baseline. The four estimates overlap heavily.

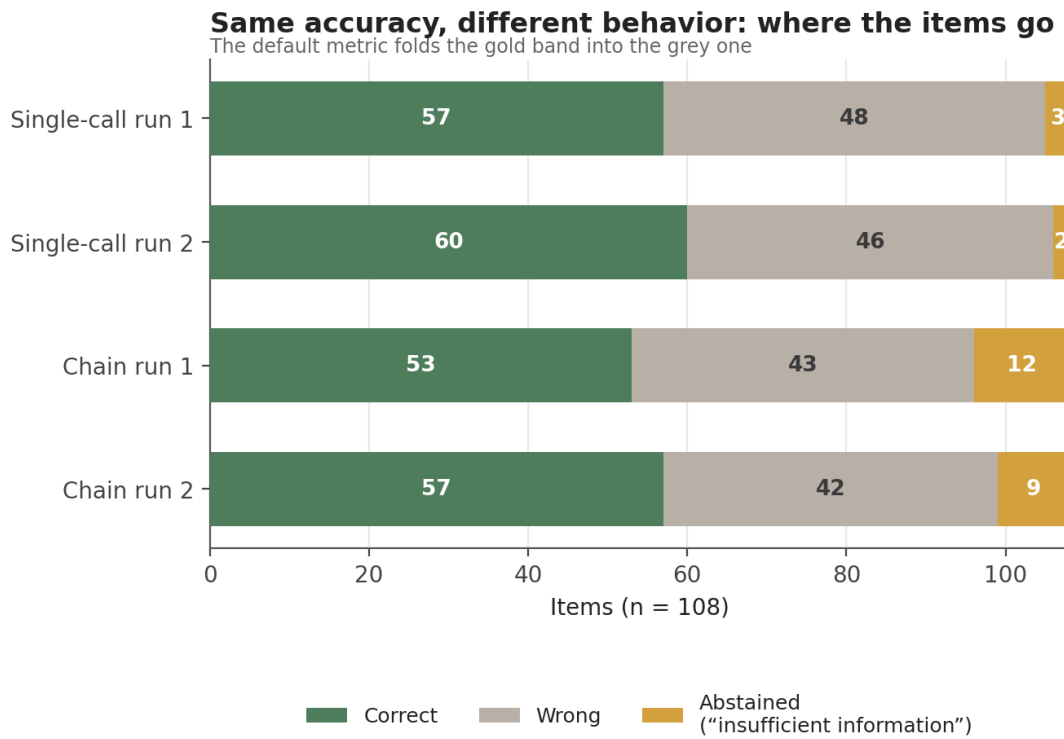


Figure 3. Outcome composition for all four runs. Correct, wrong, and abstained counts sum to 108. The gold band is what the default metric folds into the grey one.

Table 1. Per-run outcome counts. Correct and wrong counts are back-computed from the reported accuracies ($\times 108$) and recover exact integers; see the note on derived quantities at the end.

Metric	Single run 1	Single run 2	Chain run 1	Chain run 2
Correct items	57	60	53	57
Wrong items	48	46	43	42
Abstained items	3	2	12	9
Accuracy (as reported)	0.528	0.556	0.491	0.528
Precision (answered-only accuracy)	0.543	0.566	0.552	0.576
Coverage	0.972	0.981	0.889	0.917
Abstention rate	0.028	0.019	0.111	0.083
Model calls / item	1	1	5	5

Abstentions moved a lot: The single-call scaffold selected the insufficient-information option on 3 and 2 of 108 items across its two runs. The chain scaffold selected it on 12 and 9. That is roughly four times the rate, on identical questions, with the same model. The comparison is paired, since both arms see the same items in the same replicate, so an exact McNemar test is the right one: $p = 0.035$ for replicate 1 (12 chain-only abstentions vs 3 single-only) and $p = 0.016$ for replicate 2 (7 vs 0). Pooling

the two replicates as a per-item rate and clustering by item, the difference is 0.074 with a 95% interval of [0.028, 0.120]. Unlike the reproducibility gap, this one does not cross zero.

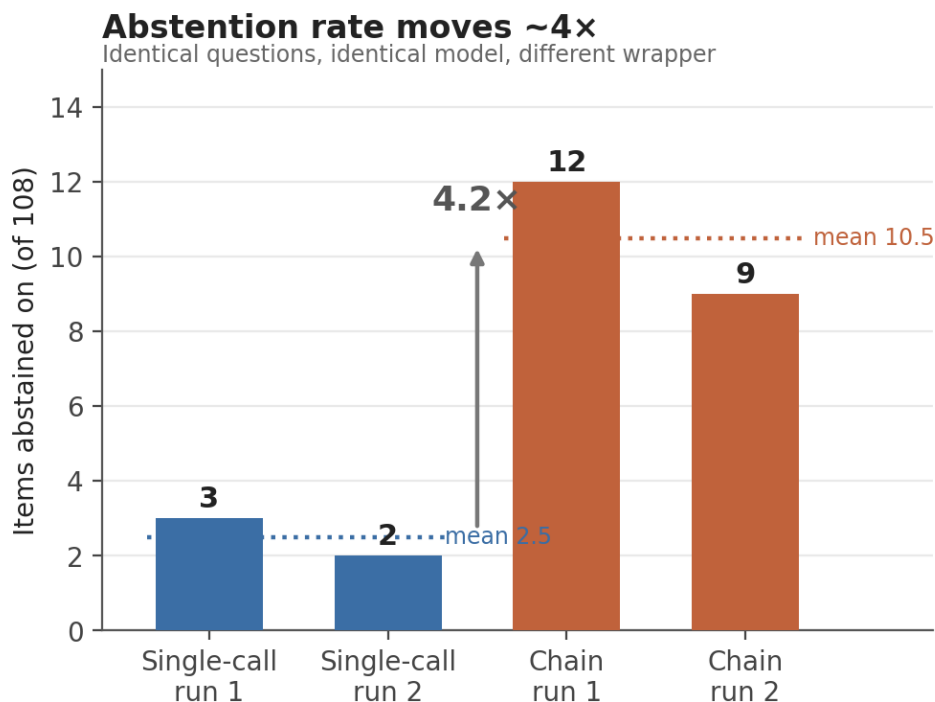


Figure 4. *In-band abstentions — the model selecting the benchmark's “insufficient information” option — on identical items. Zero parse failures across all 432 samples.*

The abstentions are not landing on unanswerable questions. Across the two chain runs there were 21 abstention events, spanning 17 distinct items. The single-call scaffold answered 16 of those 17 items normally in at least one run, and only 2 were ever abstained on by both scaffolds. The items are answerable, and the single call answers them.

They are harder than average, though. Across the 31 single-call answers on those 16 items, 11 were correct and 20 were wrong, an accuracy of 0.35 against the single call's 0.54 overall. Per item, 4 were correct in both single-call runs, 8 were wrong in both, and 3 split; the sixteenth was answered in only one run (wrong), because the single call abstained on it in the other, which is why the answer count is 31 rather than 32. This is a post hoc comparison on a subset selected by the other arm's behavior, so treat it as a lead rather than a result. It does point toward the chain scaffold abstaining more often where the model is genuinely uncertain.

Post hoc subset — selected by the other arm's behavior, so a lead, not a result

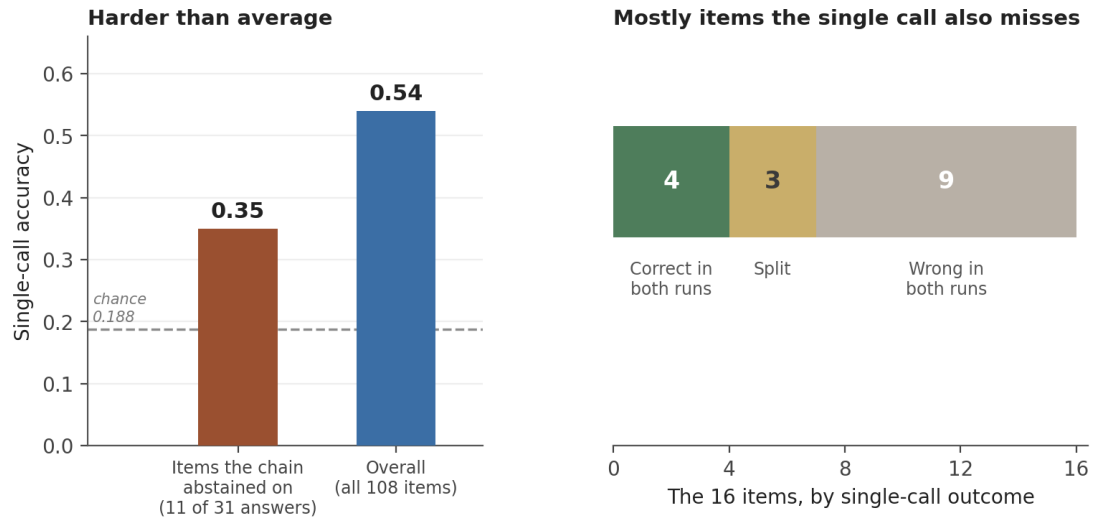


Figure 5. The 16 items the chain abstained on and the single call answered. Left: single-call accuracy on that subset versus overall. Right: per-item consistency across the two single-call runs.

They are also not landing on the same items twice. The two chain runs shared only 4 abstained items out of 12 and 9, a Jaccard overlap of 0.235. Agreement between the two scaffolds on which items trigger abstention was worse than chance in one replicate and modest in the other, kappa of -0.047 and 0.344. So the elevated abstention rate is real at the aggregate level and unstable at the item level. The chain scaffold abstains more often on items the model finds hard, but it does not identify a stable set of them.

Elevated in aggregate, unstable per item

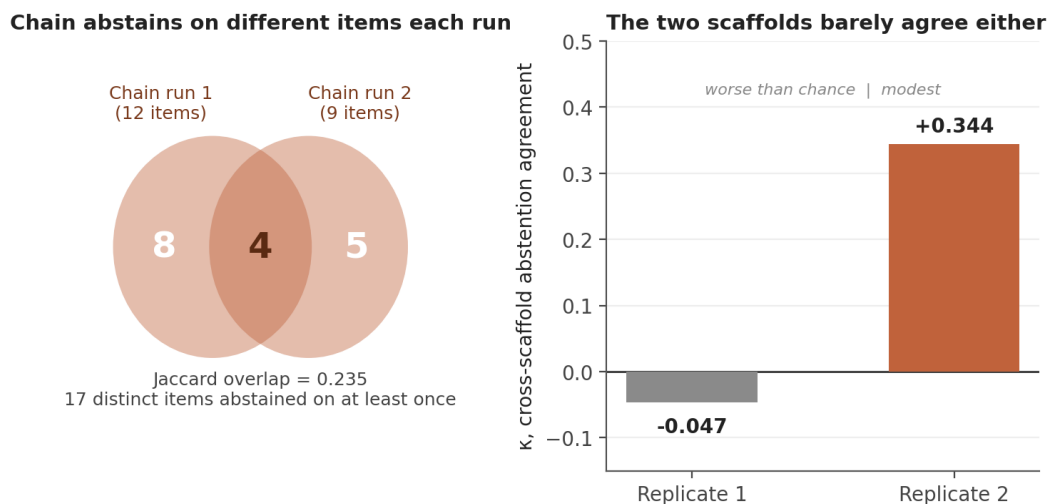


Figure 6. Left: overlap between the two chain runs' abstained-item sets. Right: cross-scaffold agreement on which items trigger abstention, by replicate.

Reproducibility looks worse under the chain scaffold, but the data cannot confirm it: Within-arm agreement across the two runs was kappa 0.588 for the single call and 0.438 for the chain, a gap of -0.150. The 95 percent bootstrap interval on that difference is [-0.356, 0.057], which crosses zero.

Restricting to items where neither run abstained, both kappas rose slightly and the gap narrowed to -0.104, interval [-0.345, 0.142]. So the reproducibility difference is not purely an artifact of abstention instability, but the residual is even less well supported.

Reproducibility: point estimate negative, interval inconclusive

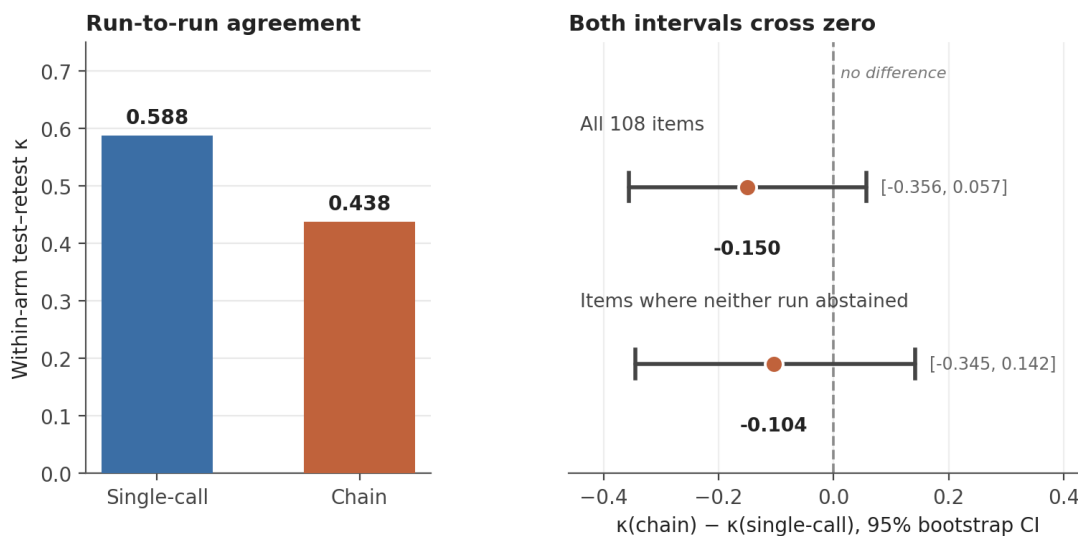


Figure 7. Left: within-arm test–retest agreement between the two runs of each scaffold. Right: the chain–single difference with 95% bootstrap intervals, under both exclusion criteria.

The point estimate is negative under both exclusion criteria. The interval crosses zero under both. This is suggestive and underpowered, not established.

What this does and does not show:

Under a fixed model on this benchmark, changing the scaffold left accuracy alone, roughly quadrupled the rate of abstention without any stable per-item pattern, and left a negative reproducibility gap that 108 items and two replicates cannot resolve.

The abstention result is the cleanest thing here, and it is worth stating carefully because it is narrower than it first looks. This is not a model refusing to engage. It is a model choosing a benchmark-provided "insufficient information" option four times as often when the same question reaches it through a decomposition chain instead of a single call. The model's willingness to commit to an answer is partly a property of the wrapper.

That has a consequence for how these scores get read. The default accuracy metric folds abstention into error, so a scaffold that shifts coverage by seven points can post an unchanged accuracy figure

while the model is doing something visibly different. LAB-Bench anticipates this: its scorer emits precision and coverage alongside accuracy, and the chain arm's precision (0.552, 0.576) is if anything slightly above the single call's (0.543, 0.566) while its coverage drops from about 0.98 to about 0.90. The problem is not that the metric doesn't exist; it is that accuracy is what gets quoted, and accuracy alone does not identify the model.

Whether this generalizes to actual refusal, meaning a model declining a request on safety grounds rather than selecting an uncertainty option, is not something these runs can address. Testing that would require a benchmark where declining is a free-text act rather than one of the answer choices.

The reproducibility question is the more interesting one and the one I cannot answer. If some scaffolds produce systematically noisier measurements than others, then a benchmark number obtained under an agentic scaffold is a less stable estimate than the same benchmark under a single call, and almost nobody reports scaffold-level test-retest reliability at all. My data leans that way and does not establish it.

Limitations

One model. Nothing here speaks to whether model rankings change with scaffold, which would require at least two models.

Abstention here is an answer choice, not a free-text decline. Every abstention was the model selecting an option the benchmark supplied. Nothing here measures refusal in the safety sense, and the two should not be equated.

Inference compute is uncontrolled. The chain scaffold made 5 calls per item to the single call's 1, one decomposition call, three sub-question calls, and one recombination call. The decomposition step yielded three usable sub-questions on all 216 chain items (the parser truncates at three, so any longer decomposition would also record as three), so the fallback path was never taken and the call count never varied at five. Any accuracy difference could be a compute effect rather than a structure effect. This matters less for the disagreement results, which ask which items flip rather than how many are right, but it is a real gap and the first thing I would fix.

Structure and context isolation vary together. The chain differs from the single call in two ways at once, decomposition and fresh contexts. I cannot attribute anything to one rather than the other. A shared context chain arm would separate them and costs almost nothing to add.

The answers are multiple choice, so the recombination step selects from a list rather than writing an answer. That is a weak operation and may have compressed any real effect toward zero. A null here means the scaffold did not matter for forced-choice selection on this benchmark, not that scaffolds do not matter.

The observation that abstentions originate at the recombination step rather than in any sub-answer came from a keyword heuristic over sub-answer text, not a formal signal. Treat it as a hunch.

What would settle it:

A power analysis run after the fact puts a confirmatory study at 8 replicates per arm if the true reproducibility gap matches the 0.15 estimate from these runs, 18 replicates if the gap is 0.10, and 72 if it is 0.05. The four runs here were exploratory and nothing was pre-registered. I am stating that plainly rather than adding replicates until the interval stopped crossing zero, which would have been optional stopping.

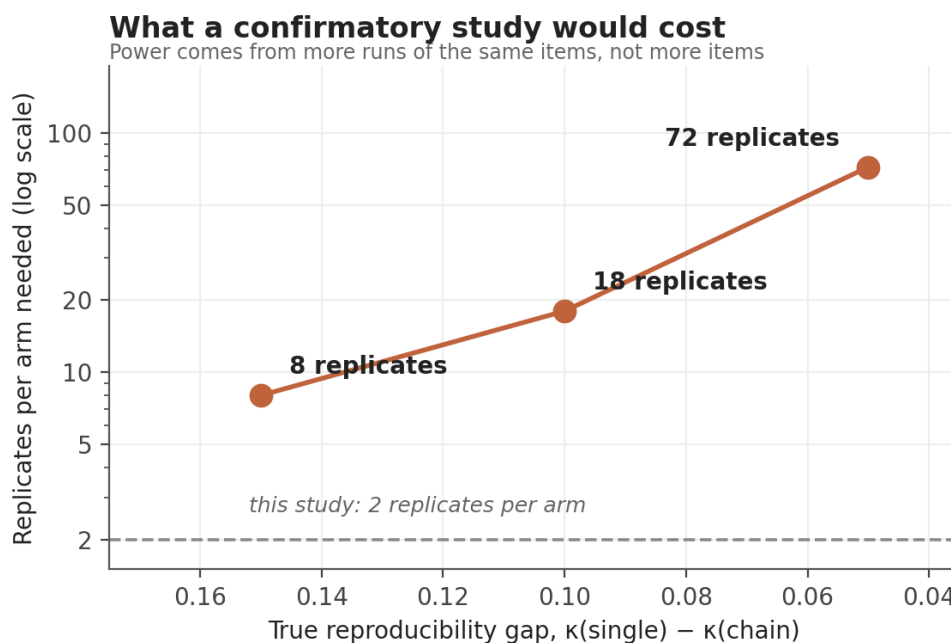


Figure 8. Replicates per arm required for a confirmatory study, as a function of the true reproducibility gap. Log scale.

A caveat on that projection, which I would rather state than have someone find. The SE it scales down is an item-level bootstrap SE: it comes from resampling the 108 items with replacement while holding each item's observed pair of outcomes fixed, so it measures how much the kappa gap would move if I had drawn a different item set, not how much it would move on a second pair of runs over the same items. Averaging P disjoint replicate-pairs on the same 108 items does not shrink that component; it is common to every pair. So $SE/\sqrt{P}$ is the wrong scaling, and 8/18/72 should be read as a floor on the replicates needed rather than a conservative estimate. The same caveat applies to the $[-0.356, 0.057]$ interval on the gap itself: it is an interval over item sets, not over runs. I have one estimate of the reproducibility gap and no direct estimate of its run-to-run variability, because that would take at least two replicate-pairs per arm and I ran one.

Code and aggregate per-item outcome labels are at github.com/santi1bent-cmd/scaffold-sensitivity-protocolqa.

Raw model transcripts are not published.

`verify_public.py` recomputes these statistics from the published labels alone, using only the Python standard library, and asserts the per-run outcome counts, the within-arm and between-arm kappas, their bootstrap intervals, and the abstention Jaccard against the values reported here.